Mentimeter

NTV 2026

Autoregressive Language Models

Lecturer: Sandro Pezzelle
module 4, lecture b

Quick updates

- We are grading! Midterm exam, in-class quiz #2, T3
- You are now working on T4 in the labs
- Today: in-class quiz #3
- After today: 2 more modules to go

Where are we at?

- We started dealing with structured data (sequences)
- NGram LMs use chain rule and a Markov assumption
- Tabular cpds are extremely inefficient for this model
- FFNNs can predict Categorical cpds efficiently

Outline

- NGram LMs
- Autoregressive LMs
- Exercises

NGram LMs

Start with chain rule

$$P(X = w_{1:l})$$
$$= \prod_{i=1}^{l} P(W = w_i | H = w_{<i})$$

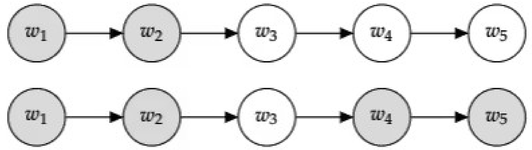
Make a Markov assumption

(cond. indep.):

given recent past $w_{i-n+1:i-1}$, the distribution of W_i is independent of $w_{1:i-n}$

$$P(X = w_{1:l})$$
$$\overset{\text{ind.}}{=} \prod_{i=1}^{l} P(W = w_i | H = w_{i-n+1:i-1})$$

Applications



Autocorrection

Autocomplete

Applications

- Autocomplete: continue a prefix string
- Autocorrection: edit parts of a given string

Markov assumptions

lead to feasible models, but are unreasonable for language modelling, severely limiting applications

Limitations of Composition via FFNNs

A feed-forward neural network maps a **fixed number of input units** to a fixed number of hidden units and then those to a fixed number of output units.

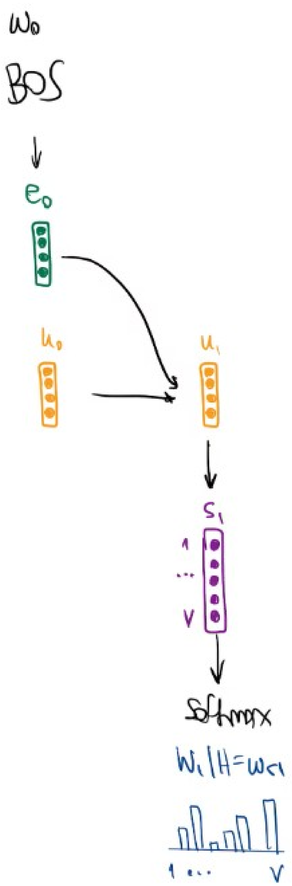
If from one step of the sequence to the next the number of inputs changes, FFNNs cannot help us.

Outline

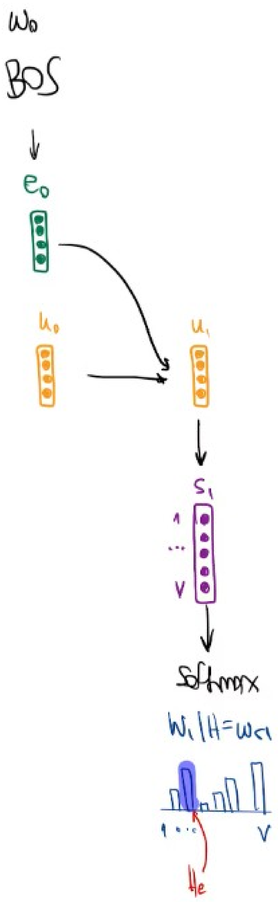
- NGram LMs
- **Autoregressive LMs**
- Exercise

A better strategy

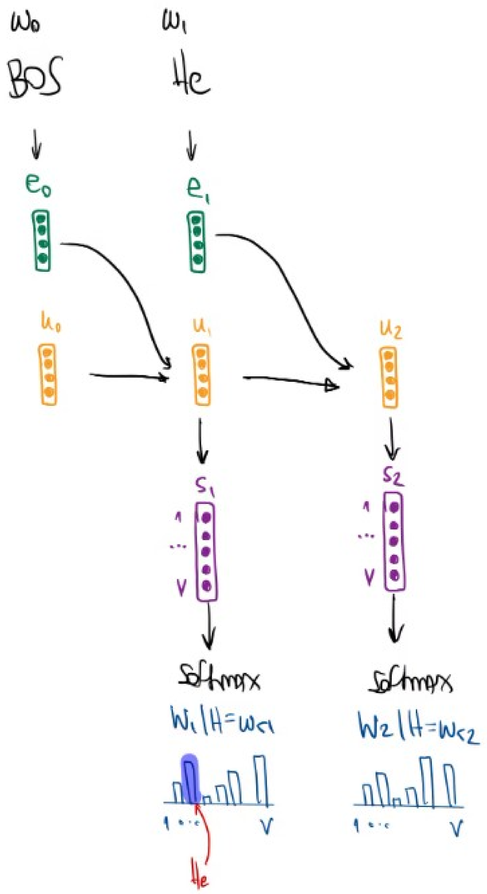
At each step of the sequence we summarise the entire sequence in a single *memory state* (or memory cell), this way the number of inputs per step will stay constant.



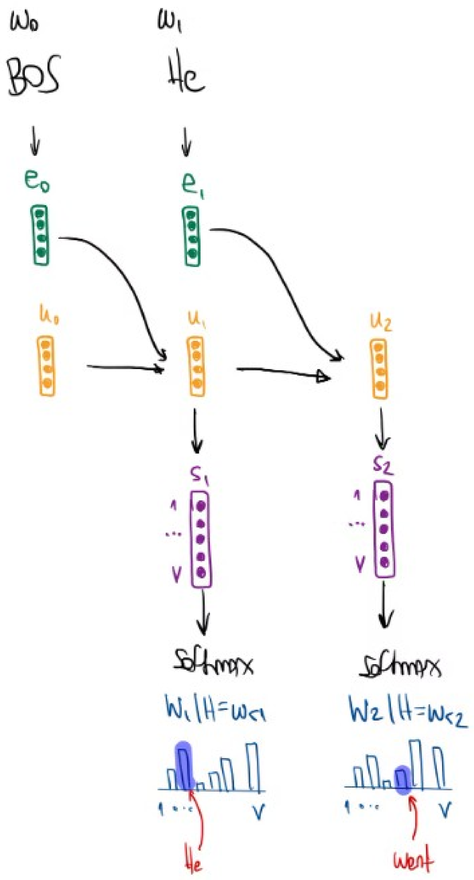
Start with an initial state $\mathbf{u}_0$. Condition on BOS and update the state to $\mathbf{u}_1$. Then, predict the distribution of the first word.



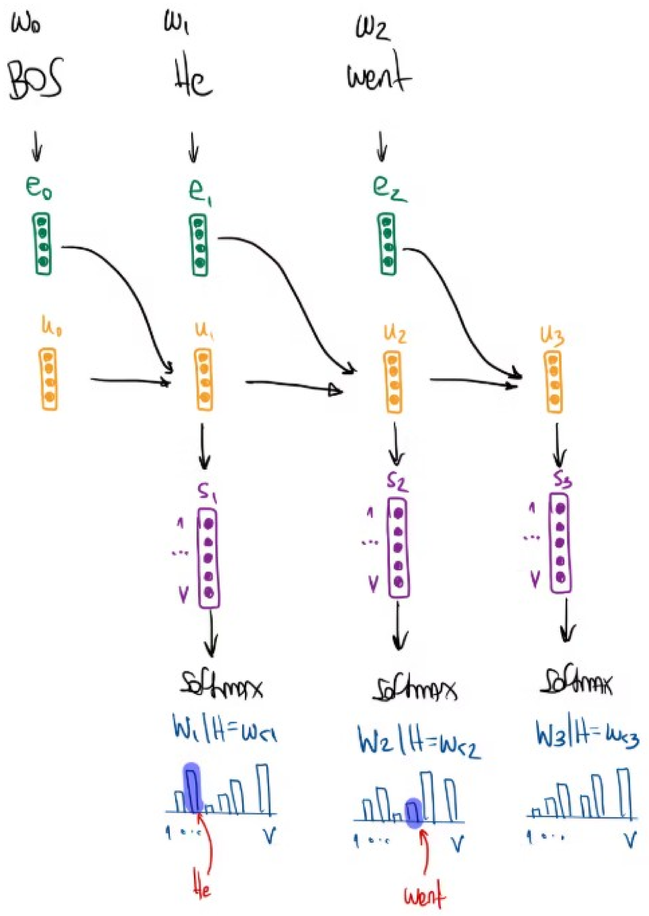
From where we draw the first word with probability $P(W = w_1|H = w_{<1})$. During training, we don't actually sample it, we _observe_ it (from a dataloader).



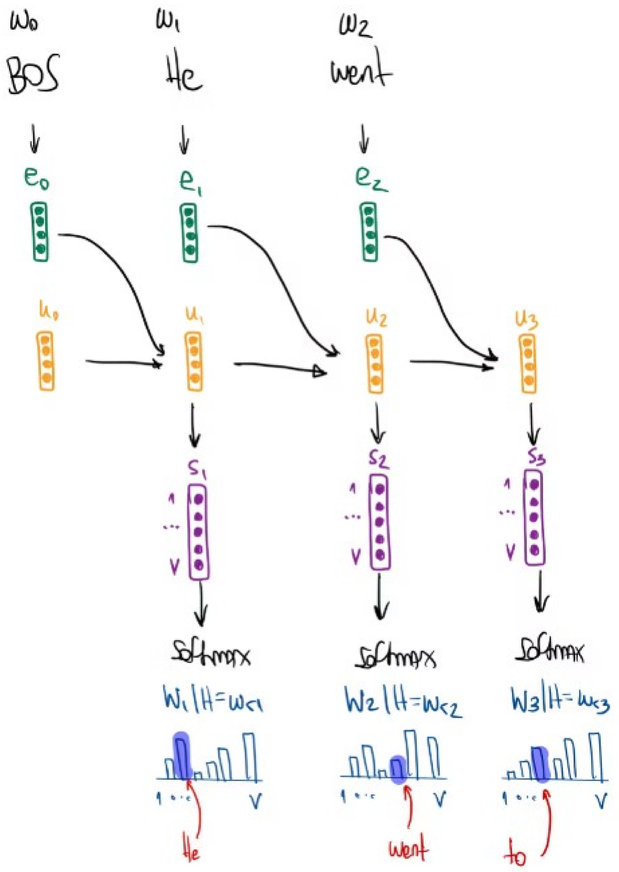
Update the state $\mathbf{u}_1$ to also condition on the newly generated/observed word. Predict the distribution of the second word from $\mathbf{u}_2$.



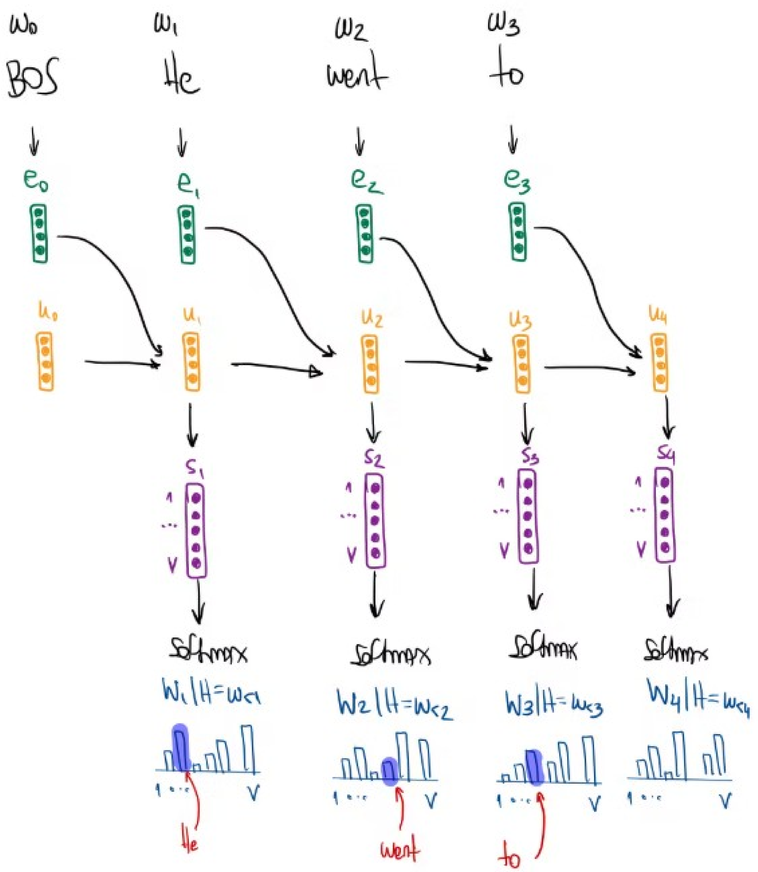
From where we draw the second word with probability $P(W = w_2 | H = w_{<2})$. Again, during training we don't draw it, we _observe_ it.



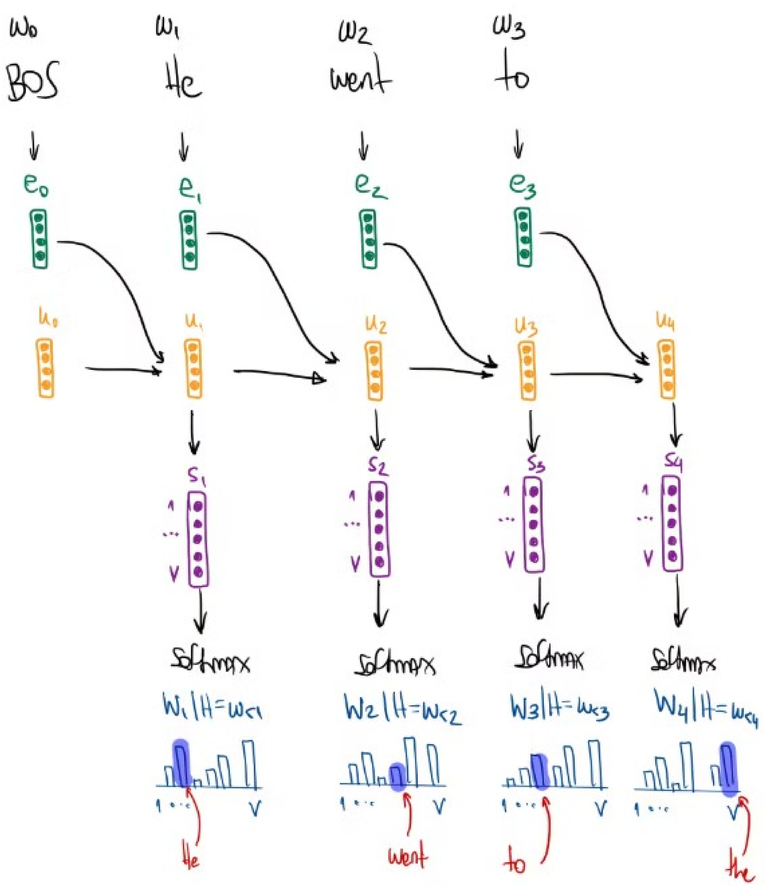
Update the state $\mathbf{u}_2$ to also condition on the newly generated/observed word. Predict the distribution of the third word from $\mathbf{u}_3$.



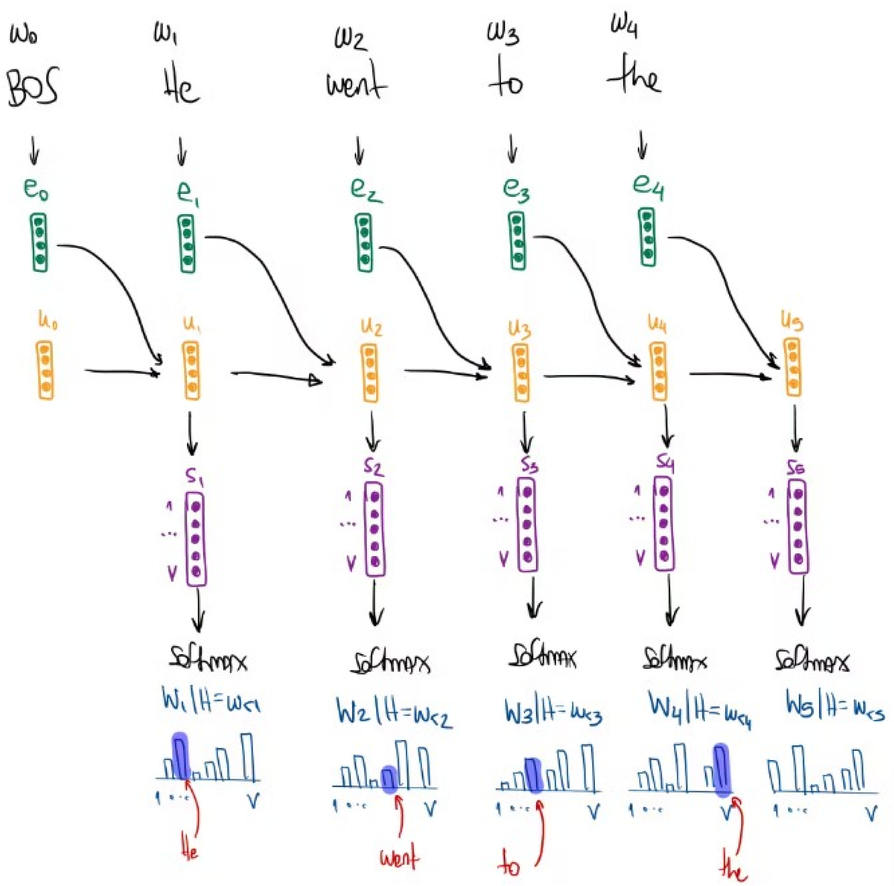
From where we draw the third word with probability $P(W = w_3 | H = w_{<3})$.



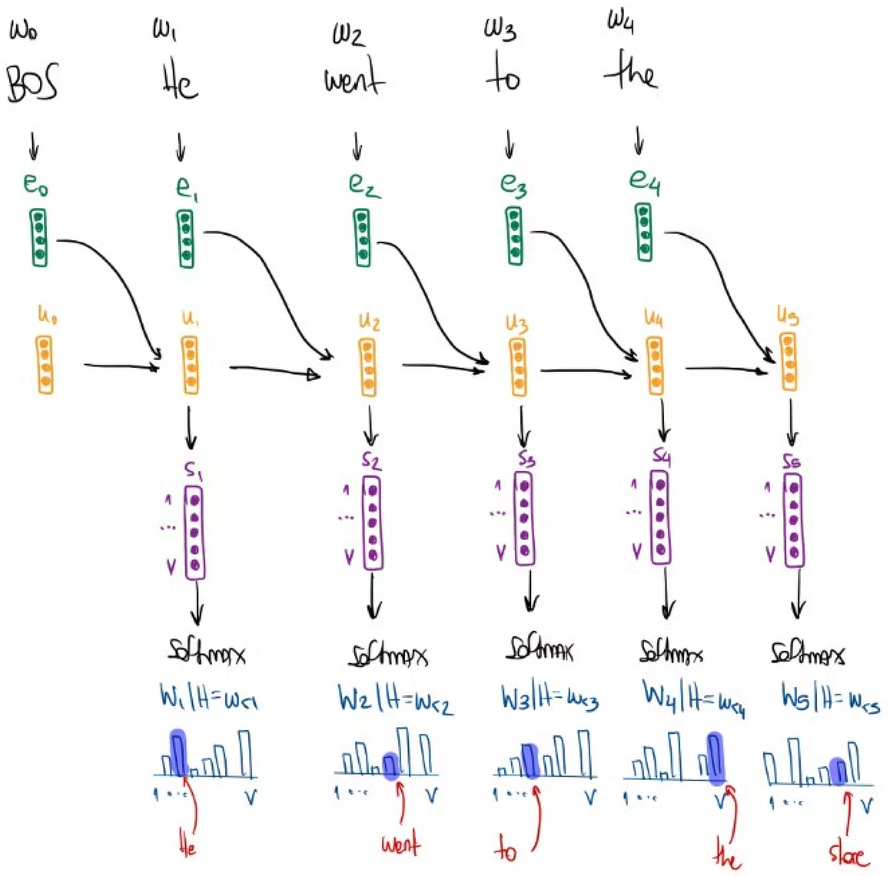
Keep reusing this same mechanism, that is, predict the distribution of the fourth word given full history.



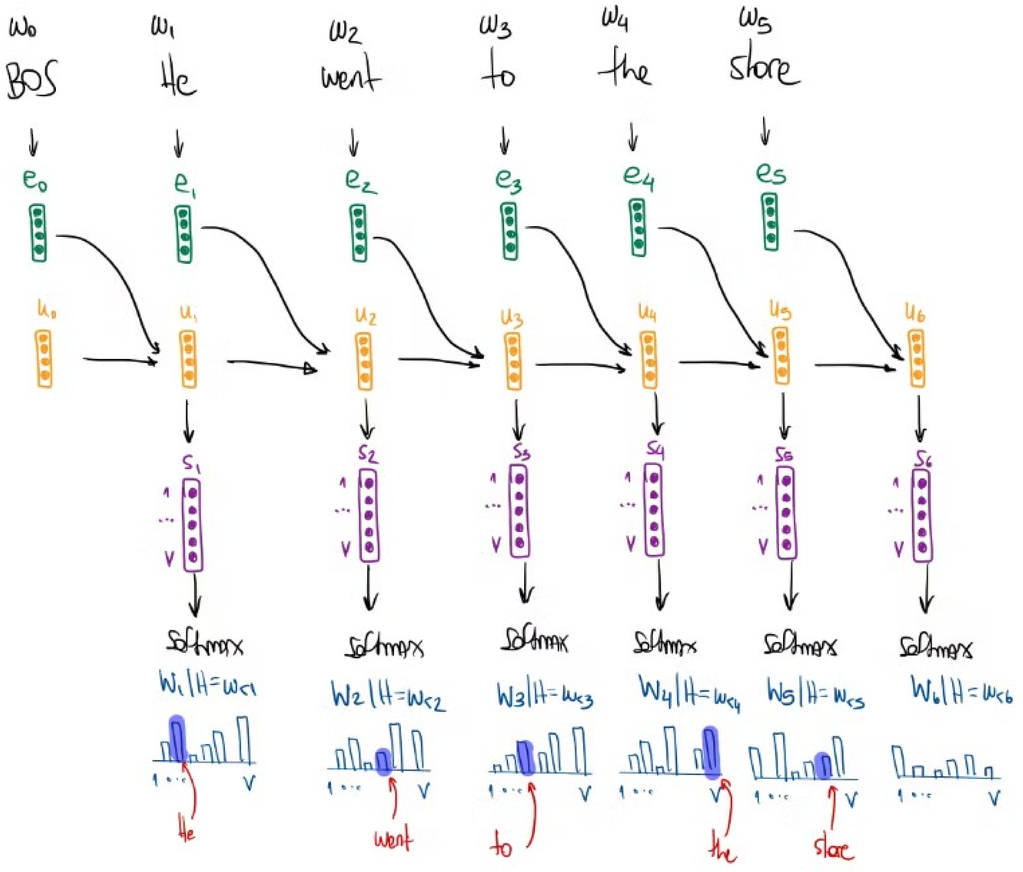
From where we draw the fourth word with probability $P(W = w_4 | H = w_{<4})$



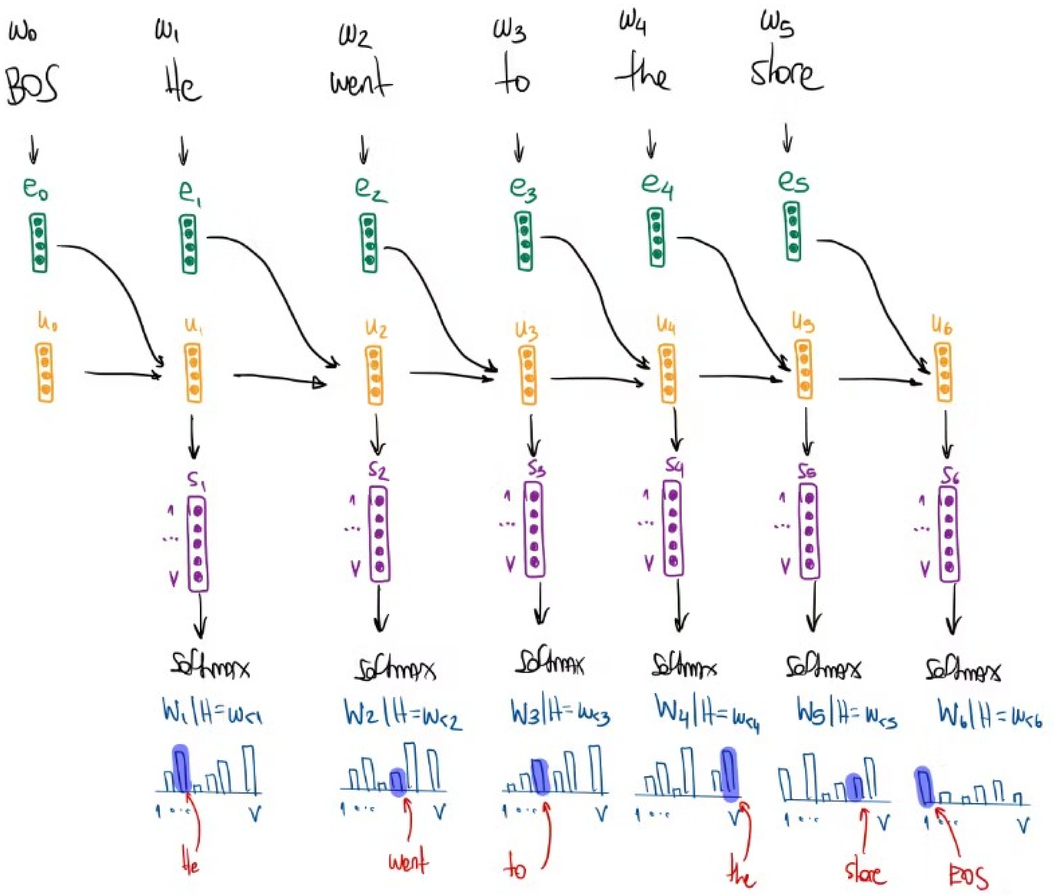
We always update the state using 2 inputs (previous state and previous word), the new state represents the generation context.



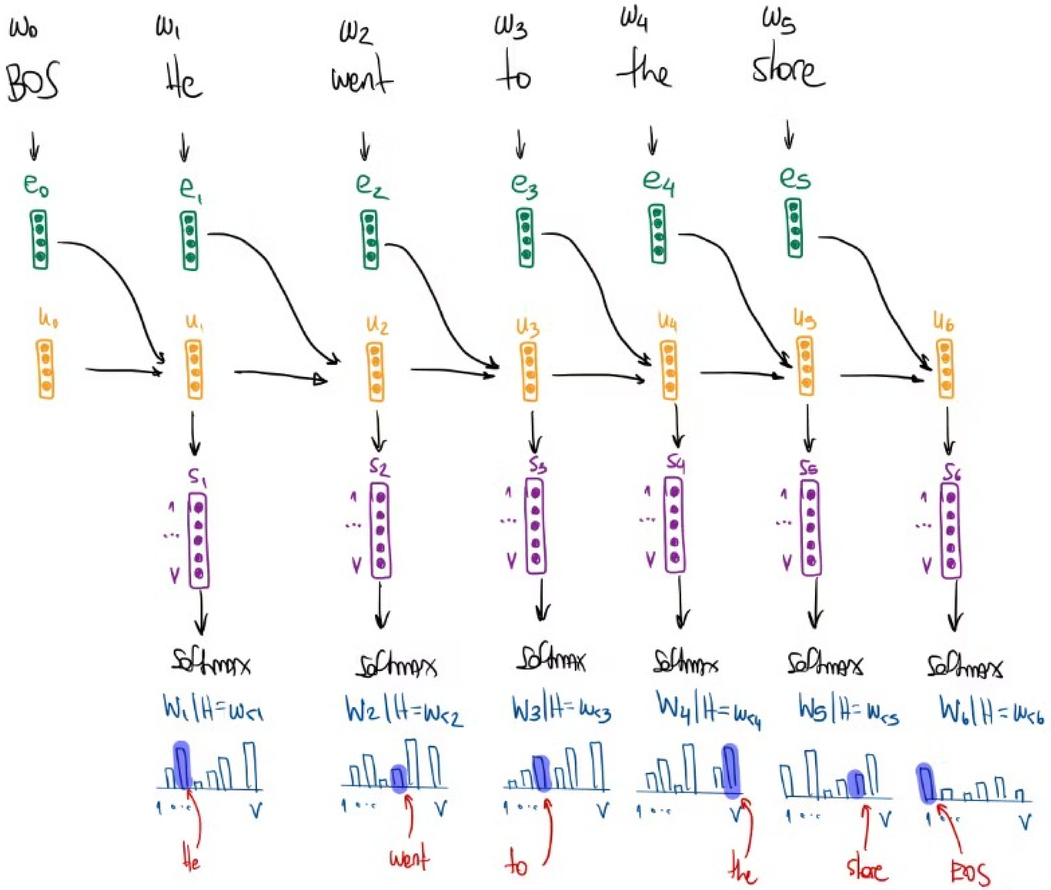
When we predict the distribution of W_5 using $\mathbf{u}_5$, we are conditioning on $w_{<5}$.



The mechanism has fixed dimensionality, but it can be applied *recursively* along the sequence as it grows.



Until we draw a symbol (EOS) that interrupts this recursion.



Recurrent neural network (step)

We start from some initial H-dimensional state $\mathbf{u}_0 = \mathbf{0}$ and update it recursively:

$$\mathbf{u}_i = \mathrm{rnnstep}_H(\mathbf{u}_{i-1}, \mathbf{e}_{i-1}; \theta_{\mathrm{step}})$$

This update function is a simple nonlinear transformation. Note that no matter the step i, it always takes the same number of inputs (one H dimensional vector and one D dimensional vector).

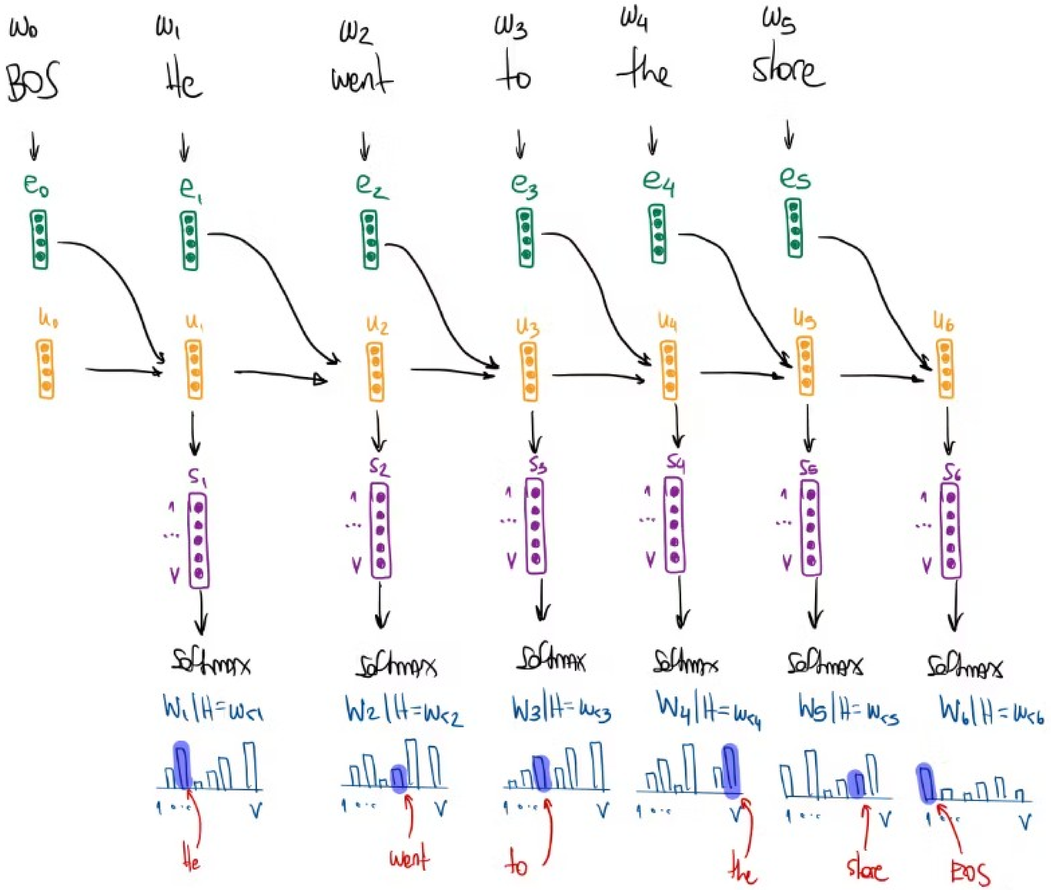
Simplest RNN step

The simplest RNN cell is a single non-linear layer:

$$\mathbf{u}_i = \tanh(\mathbf{R}\mathbf{u}_{i-1} + \mathbf{W}\mathbf{e}_{i-1})$$

with $\mathbf{R} \in \mathbb{R}^{H \times H}$ and $\mathbf{W} \in \mathbb{R}^{H \times D}$

The non-linearity is important, without it, the result is not affected by the order of the inputs.



Output Layer

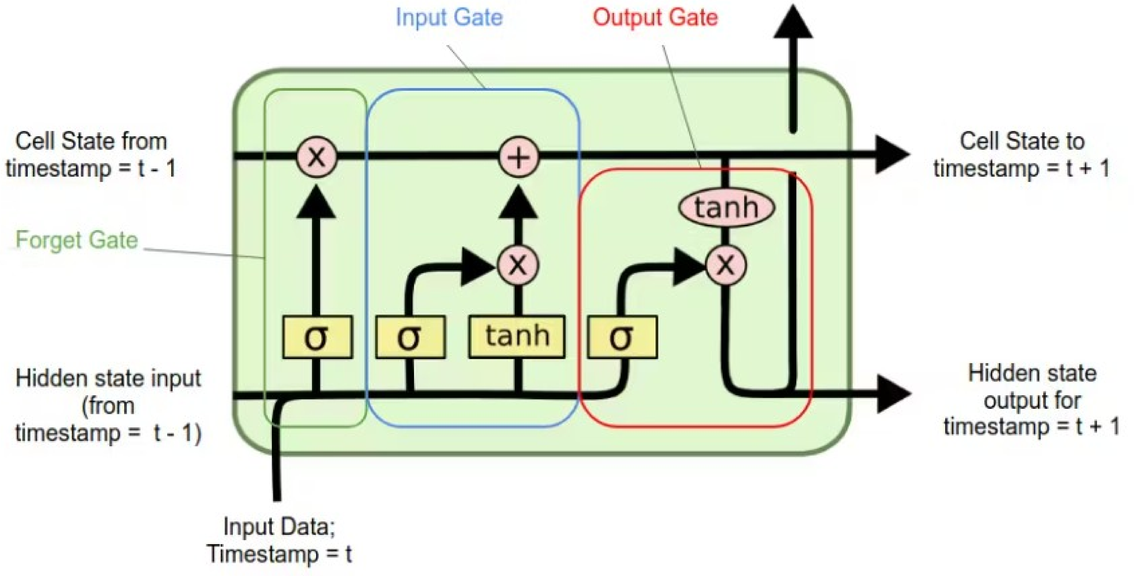
Each state $\mathbf{u}_i$ depends on the sequence before w_i, we can turn that state into logits

$$\mathbf{s}_i = \mathrm{linear}_V(\mathbf{u}_i; \theta_{\mathrm{out}})$$

and use those logits to parameterise a cpd without Markov assumption:

$$W_i | H = w_{<i} \sim \mathrm{Cat}(\mathrm{softmax}(\mathbf{s}_i))$$

LSTM



- **Cell State:** A "conveyor belt": it carries information throughout the entire sequence with only minor linear interactions, making it easy for data to flow unchanged.

- **Forget Gate:** Decides which information from the previous state is no longer relevant and should be discarded.

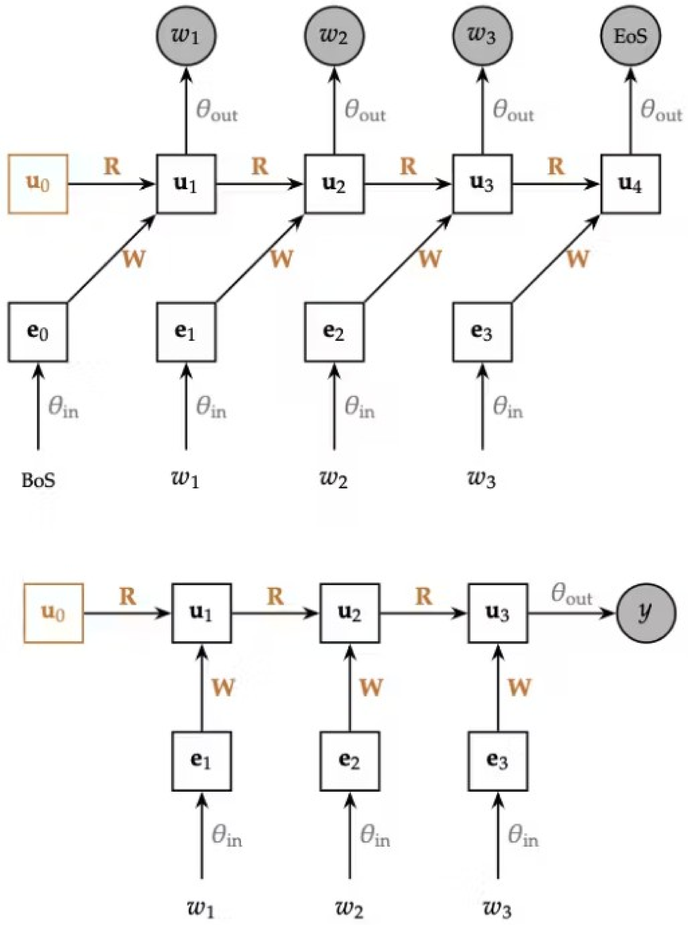
- **Input Gate:** Determines what new information from the current input is worth storing in the cell state.

- **Output Gate:** Decides which part of the current cell state will be output to the next step.

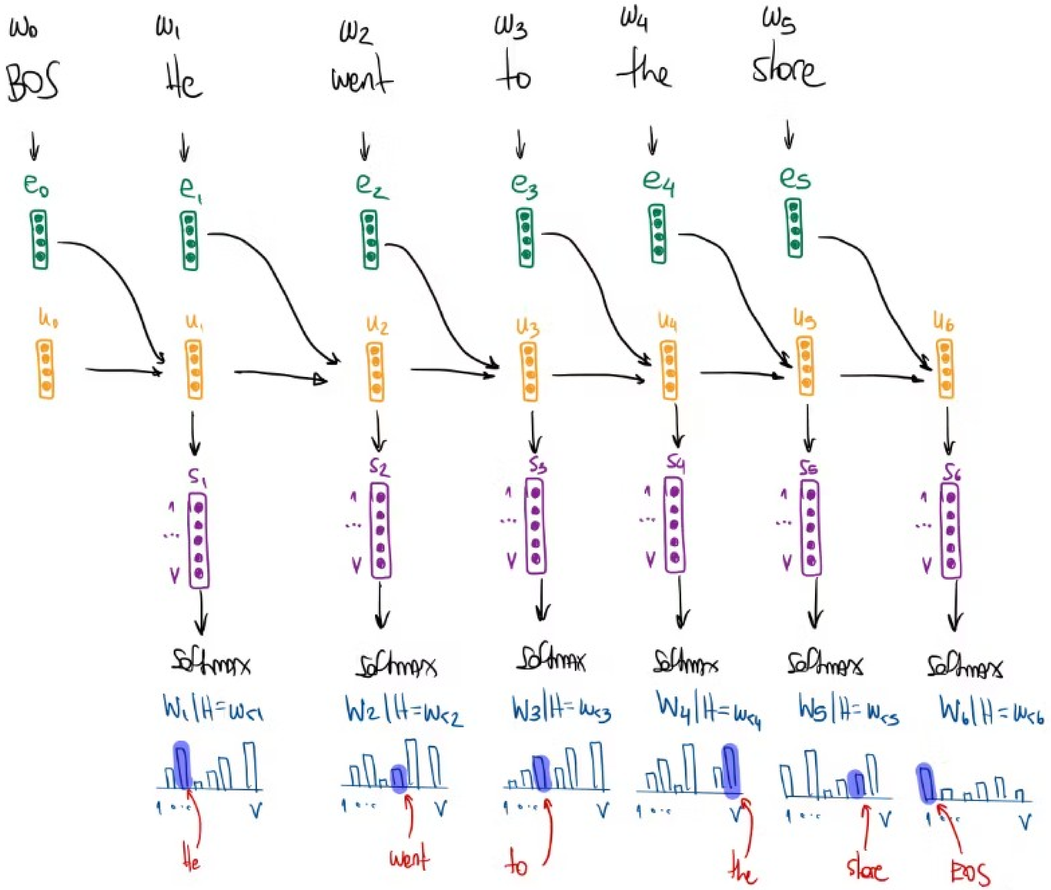
Autoregressive Language Model

Any language model that uses the
entire ordered history to parameterise
the distribution of the next word:

$$W|H = w_{<i} \sim \mathrm{Cat}(\boldsymbol{\pi}(w_{<i}; \theta))$$



Top: RNN *decoder* (used in LMs). Bottom: RNN *encoder* (used in classifiers, or for conditioning on a sequence that we do not need to generate).



Training

We train the model by maximising the log-likelihood of the NN parameters given a dataset of observed sentences.

For one sentence $w_{1:l}$, we choose θ to maximise

$$\sum_{i=1}^{l} \log \mathrm{Cat}(w_i | \boldsymbol{\pi}(w_{<i}; \theta))$$

via gradient-based search.

With a mini batch of S sentences, we would average their log-probabilities.

Generation by sampling

1. We start with H = <BOS>

2. Condition on the current history and predict the distribution of the next word.

3. Draw an outcome from that Categorical cpd, and extend the history with the draw.

4. If we got EOS in (3), we stop; otherwise, we repeat from (2).

This is also known as **ancestral sampling**.

Applications

- Conditional generation
  - machine translation
  - question answering
  - summarisation
  - essay/story generation
- Conversational agents
  - customer service
  - personal assistants

Outline

- NGram LMs
- Autoregressive LMs
- **Exercises**

Mentimeter

Compositions functions for an autoregressive LM

Average pooling: h[i] = avg(e[0], ..., e[i-1])

Concatenation: h[i] = concatenate(e[0], ..., e[i-1])

RNN decoder: h[i] = rnnstep(h[i-1], ..., e[i-1])

RNN encoder: h[i] = rnnstep(h[i-1], ..., e[i])

Bidirectional RNN encoder: h[1], ..., h[L] = birnn(e[1], ..., e[L])

Nay

Yay

Solution

A (nay). $\mathbf{h}_i = \mathrm{avg}(\mathbf{e}_0, \ldots, \mathbf{e}_{i-1})$
the state cannot capture word order.

B (nay). $\mathbf{h}_i = \mathrm{concat}(\mathbf{e}_0, \ldots, \mathbf{e}_{i-1})$
variable-size state.

C (*yay*). $\mathbf{h}_i = \mathrm{rnnstep}(\mathbf{h}_{i-1}, \mathbf{e}_{i-1})$
the state depends on ordered past information.

D (nay). $\mathbf{h}_i = \mathrm{rnnstep}(\mathbf{h}_{i-1}, \mathbf{e}_i)$
the state depends on information not yet available
(we have not yet generated the ith word).

E (nay). $\mathbf{h}_{1:L} = \mathrm{birnn}(\mathbf{e}_{1:L})$
use $w_i \cdots w_L$ before they have been generated.

What did you learn today?

Summary

- Recurrent NNs
- Autoregressive LMs

What next?

- Sequence labelling: classification and generation meet